



```
In [3]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
```

```
In [14]: # Load data

columns = ['letter'] + [f'feature_{i}' for i in range(1,17)]

df = pd.read_csv("letter-recognition.csv")

print(df.head())
```

	letter	xbox	ybox	width	height	onpix	xbar	ybar	x2bar	y2bar	\
0	T	2	8	3	5	1	8	13	0	6	
1	I	5	12	3	7	2	10	5	5	4	
2	D	4	11	6	8	6	10	6	2	6	
3	N	7	11	6	6	3	5	9	4	6	
4	G	2	1	3	1	1	8	6	6	6	

	xybar	x2ybar	xy2bar	xedge	xedgey	yedge	yedgex
0	6	10	8	0	8	0	8
1	13	3	9	2	8	4	10
2	10	3	7	3	7	3	9
3	4	4	10	6	10	2	8
4	6	5	9	1	7	5	10

```
In [15]: X = df.iloc[:, 1:].values
y = df.iloc[:, 0].values
```

```
In [16]: le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

```
In [17]: y_categorical = to_categorical(y_encoded)
```

```
In [18]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [19]: X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_categorical, test_size=0.2, random_state=42
)
```

```
In [20]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

model = Sequential()

model.add(Input(shape=(16,))) # ☒ Correct way
model.add(Dense(128, activation='relu'))
```

```
model.add(Dense(64, activation='relu'))  
model.add(Dense(26, activation='softmax'))
```

```
In [21]: model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
In [22]: history = model.fit(  
    X_train, y_train,  
    epochs=30,  
    batch_size=32,  
    validation_split=0.2  
)
```

Epoch 1/30
400/400  **1s** 817us/step - accuracy: 0.5692 - loss: 1.5788 - val_accuracy: 0.7544 - val_loss: 0.8925

Epoch 2/30
400/400  **0s** 679us/step - accuracy: 0.7888 - loss: 0.7299 - val_accuracy: 0.8228 - val_loss: 0.6424

Epoch 3/30
400/400  **0s** 685us/step - accuracy: 0.8406 - loss: 0.5566 - val_accuracy: 0.8462 - val_loss: 0.5256

Epoch 4/30
400/400  **0s** 678us/step - accuracy: 0.8655 - loss: 0.4567 - val_accuracy: 0.8641 - val_loss: 0.4558

Epoch 5/30
400/400  **0s** 666us/step - accuracy: 0.8845 - loss: 0.3896 - val_accuracy: 0.8822 - val_loss: 0.3928

Epoch 6/30
400/400  **0s** 655us/step - accuracy: 0.8969 - loss: 0.3380 - val_accuracy: 0.8956 - val_loss: 0.3432

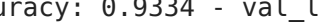
Epoch 7/30
400/400  **0s** 666us/step - accuracy: 0.9111 - loss: 0.2965 - val_accuracy: 0.9009 - val_loss: 0.3211

Epoch 8/30
400/400  **0s** 669us/step - accuracy: 0.9193 - loss: 0.2633 - val_accuracy: 0.9112 - val_loss: 0.2934

Epoch 9/30
400/400  **0s** 666us/step - accuracy: 0.9258 - loss: 0.2387 - val_accuracy: 0.9206 - val_loss: 0.2770

Epoch 10/30
400/400  **0s** 656us/step - accuracy: 0.9360 - loss: 0.2130 - val_accuracy: 0.9259 - val_loss: 0.2532

Epoch 11/30
400/400  **0s** 652us/step - accuracy: 0.9395 - loss: 0.1963 - val_accuracy: 0.9278 - val_loss: 0.2370

Epoch 12/30
400/400  **0s** 658us/step - accuracy: 0.9445 - loss: 0.1803 - val_accuracy: 0.9334 - val_loss: 0.2295

Epoch 13/30
400/400  **0s** 672us/step - accuracy: 0.9498 - loss: 0.1638 - val_accuracy: 0.9287 - val_loss: 0.2265

Epoch 14/30
400/400  **0s** 666us/step - accuracy: 0.9551 - loss: 0.1525 - val_accuracy: 0.9306 - val_loss: 0.2153

Epoch 15/30
400/400  **0s** 665us/step - accuracy: 0.9563 - loss: 0.1415 - val_accuracy: 0.9347 - val_loss: 0.2048

Epoch 16/30
400/400  **0s** 662us/step - accuracy: 0.9602 - loss: 0.1321 - val_accuracy: 0.9378 - val_loss: 0.1979

Epoch 17/30
400/400  **0s** 668us/step - accuracy: 0.9655 - loss: 0.1195 - val_accuracy: 0.9353 - val_loss: 0.1955

Epoch 18/30
400/400  **0s** 657us/step - accuracy: 0.9663 - loss: 0.1131 - val_accuracy: 0.9419 - val_loss: 0.1951

Epoch 19/30
400/400  **0s** 657us/step - accuracy: 0.9673 - loss: 0.1066 - val_accuracy: 0.9444 - val_loss: 0.1826
Epoch 20/30
400/400  **0s** 660us/step - accuracy: 0.9684 - loss: 0.1015 - val_accuracy: 0.9381 - val_loss: 0.1862
Epoch 21/30
400/400  **0s** 679us/step - accuracy: 0.9712 - loss: 0.0951 - val_accuracy: 0.9459 - val_loss: 0.1781
Epoch 22/30
400/400  **0s** 650us/step - accuracy: 0.9735 - loss: 0.0904 - val_accuracy: 0.9434 - val_loss: 0.1813
Epoch 23/30
400/400  **0s** 654us/step - accuracy: 0.9735 - loss: 0.0843 - val_accuracy: 0.9516 - val_loss: 0.1628
Epoch 24/30
400/400  **0s** 665us/step - accuracy: 0.9773 - loss: 0.0797 - val_accuracy: 0.9488 - val_loss: 0.1704
Epoch 25/30
400/400  **0s** 651us/step - accuracy: 0.9784 - loss: 0.0720 - val_accuracy: 0.9509 - val_loss: 0.1658
Epoch 26/30
400/400  **0s** 696us/step - accuracy: 0.9788 - loss: 0.0717 - val_accuracy: 0.9484 - val_loss: 0.1587
Epoch 27/30
400/400  **0s** 666us/step - accuracy: 0.9789 - loss: 0.0686 - val_accuracy: 0.9453 - val_loss: 0.1772
Epoch 28/30
400/400  **0s** 659us/step - accuracy: 0.9814 - loss: 0.0617 - val_accuracy: 0.9503 - val_loss: 0.1671
Epoch 29/30
400/400  **0s** 676us/step - accuracy: 0.9816 - loss: 0.0606 - val_accuracy: 0.9516 - val_loss: 0.1594
Epoch 30/30
400/400  **0s** 676us/step - accuracy: 0.9833 - loss: 0.0562 - val_accuracy: 0.9478 - val_loss: 0.1682

```
In [23]: loss, accuracy = model.evaluate(X_test, y_test)
         print("Test Accuracy:", accuracy)
```

125/125  **0s** 526us/step - accuracy: 0.9513 - loss: 0.1539
Test Accuracy: 0.9512500166893005

```
In [24]: pred = model.predict(X_test)

         # Convert probabilities → class index
         pred_classes = np.argmax(pred, axis=1)

         # Convert back to letters
         pred_letters = le.inverse_transform(pred_classes)

         print(pred_letters[:10])
```

125/125  **3s** 21ms/step
['X' 'L' 'A' 'E' 'Q' 'E' 'O' 'Q' 'G' 'O']

In []: